

Executable applications

Persistent objects, exact workflows and measured configuration design

Shared sequences, discrete calibration, conditional graph sampling, local geometry, combinatorial sizes, mathematical tasks and a persistent query service.

6

persistent object kinds

256

grouped arithmetic tasks

28

measured kernel trials

Tested standard-library application integrations. The conventional kernel wins the demo; global geometry and physical derivations remain open.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

What this release adds

PerfectPower now connects its exact arithmetic engines to reusable application objects: a persistent catalogue and query service, a shared sequence library, discrete calibration, conditional graph sampling, local geometry exploration, combinatorial size selection, grouped mathematical tasks and measured configuration tuning. The earlier ExactPopulation layer supplies original-object identity, huge finite counts, reversible ranks and seeded sampling. This release turns those capabilities into workflows with portable definitions, repeatable queries and concrete results.

The application runtime uses the Python standard library. SQLite holds definitions and aliases; it does not materialize every member of a large mathematical population. Six object kinds share one versioned JSONL service. Arithmetic engines retain their completeness conditions, budgets and proof provenance. This release adds application code and Python checks, with no new Lean theorem.

Persistent catalogue and query protocol

Catalogue stores population, sequence, inverse, graph, geometry and combinatorial definitions. A content address hashes the schema version, kind and canonical JSON definition. Aliases resolve to those addresses. Registering the same definition reuses its address. Replacing an alias requires an explicit replace flag, and the old definition remains accessible. Transactions prevent a failed alias replacement from inserting an orphan definition. A bounded LRU cache reuses compiled engines; reopening the database recompiles the small definitions.

The protocol accepts register, list, definition, call, restrict, compare, reweight and join operations. Responses carry request IDs. Errors occupy their own response line, allowing subsequent requests to execute. Input lines have a one-megabyte byte budget; overlong lines are drained before continuing. Methods come from a fixed allowlist. Mathematical expressions use the existing structured language. The service never evaluates supplied Python code.

Rank joins pair objects at the same ordinal position within a bounded window. They assert positional correspondence, not shared mathematical meaning. Value joins are complete equality joins over small finite populations and preserve duplicate projected values as distinct configuration identities. Both inputs and the result must fit the row budget; oversized joins fail without returning a partial answer. There is no implicit Cartesian join over enormous populations.

These semantics suit local research clients, pipelines, notebooks and embedded processes repeatedly querying the same definitions. They provide a compact executable database with query bindings. They do not replace an unrestricted SQL optimizer over arbitrary integer equations.

```
export PYTHONPATH=python
python -m perfectpower service --database /tmp/perfectpower.sqlite \
  < receipts/applications/service_requests.jsonl
```

The transcript registers all six kinds, then selects a compatible tile, executes a sequence at index 10^{15} modulo 1,000,000,007, solves a calibration target, samples a conditioned graph ensemble, bounds a local path and counts square-triangular sizes below 10^{100} . Exact outputs are in `service_replay.json`.

Shared sequence execution and executable laws

SequenceLibrary accepts one integer operator, one seed and named readouts. It compiles the readouts together through the existing integral machine and witness resolvent. A common state supports all outputs. The demo defines Fibonacci, the next Fibonacci number and Lucas numbers from a two-state transition. Forty term vectors are checked against literal integer iteration. Huge-index modular execution uses the integral machine, including moduli for which rational coordinate denominators would be unsuitable.

Generating functions are executable laws of the supplied matrix/readout definitions. Library comparisons certify equality at every nonnegative index or return the first differing index. Arithmetic subsequences compile offset and step through exact matrix powers. These results do not establish external sequence identification from a short observed prefix; the definitions are the premises.

The experiment interface compares two supplied states. It reports equality under every future readout or selects a separating readout and chronological operator word. Given exact operator and readout costs, it chooses the cheapest separator among the certificate's finite witness words. Actual left and right values make the experiment replayable. The optimum concerns that finite witness set, rather than all conceivable words. State membership concerns the existing saturated reachable space, rather than membership on the seed orbit.

For recurrence services, model diagnostics and exact signal processing, the direct capability is several executable outputs sharing one compiled state and an exact comparison interface. Conventional recurrence loops remain appropriate for small indices. No primitive-call minimum or general speedup is asserted.

Integer inverse design and discrete calibration

InverseDesign reuses an exact Smith decomposition and positive-definite metric across observations and targets. Its solve operation returns every closest integer setting satisfying the exact linear observation. The demo models three signed integer actuator increments subject to two readouts: total increment and a weighted readout. The target is rational, so objective ties remain exact.

The receipt exposes original settings, minimum energy, continuous relaxation, integer kernel and proof packet. An independent variable-elimination scan checks the demo optimum; the engine's complete bounded ellipsoid enumeration supplies the global result. Tests also compare multiple targets and tied minimizers against a separate exhaustive search. Integer-infeasible observations return a complete infeasibility result through the Smith engine.

This creates discrete designs and calibration settings directly. The model has exact linear observations and a positive-definite quadratic distance from a target. It does not impose arbitrary inequalities such as nonnegative actuator values, and the demo is not a measured physical device. Users supply the application matrix and metric. Lattice enumeration remains exponential with explicit budgets.

Exact graph ensemble sampling

GraphEnsemble samples determinant-weighted bases of a cyclic gain graph. It visits edges in order. At each undecided edge it divides an exact joint event probability by the exact probability of decisions already made, then uses an integer random draw for that rational Bernoulli probability. The result is a full-rank basis with its conditional probability and every decision recorded. Sampling does not enumerate all bases.

Cyclotomic orders 2, 3, 4 and 6 are supported because their real event probabilities are rational. Higher orders are rejected pending exact algebraic comparison. Samples are independent with replacement. They are uniform only when the supplied determinant weights make the basis distribution uniform. Include/exclude constraints use the same event semantics. Zero-probability conditions and rank-deficient measures are rejected. An aggregate event-work estimate limits sampling.

The demo emits 32 unconditional configurations, 32 with an excluded edge, and eight after a weight repair. It uses a two-vertex signed gain model with loops and parallel edges. Its full-rank bases are gain-graph configurations, rather than ordinary unsigned spanning trees. Weight changes use the existing low-rank repair engine. Derived objects preserve the source definition and include an update proof.

One test exhausts every random draw path on a weighted three-choice graph and sums exact rational probability mass. Another compares multivertex event marginals and sampled basis probabilities with an independent forest-coefficient basis sum. Correctness is checked by exact distribution identities, rather than an empirical histogram.

The direct application is stochastic network configuration generation for a specified gain-graph model and condition. These probabilities are not probabilities of integer solutions and do not establish a real-world network model without domain assumptions.

Offline geometry workbench

GeometryWorkbench compiles named local metric panels and reuses each checked packet for point and contained segment queries. The demo opens branch, infinity and finite panels for $y^2 = x^3 - x$. Points report rational density enclosures. Straight segments report squared path-length intervals from certified tangent metric comparisons.

The generated `geometry_workbench.html` is an offline interactive client. Panel selection and two grid controls inspect 81 points per panel and paths from the lower corner. Colour uses approximate interval midpoints; numerical readouts preserve rational bounds. Data are embedded, so no service or network is needed. `geometry_panels.json` carries the full packets and smaller grids for machine consumers.

These are independent local panels. This release does not discover transition maps or claim a compatible global atlas, unrestricted geodesic distances, global smooth Voronoi geometry or a period-normalized Bergman metric. The underlying metric is the stated sum of squared differential norms. The direct result is interactive certified local calculation.

Combinatorial size design

CombinatorialDesign connects a chosen fixed-width Gamma/binomial expression to the domain-aware power compiler and square-triangular size queries to the complete Pell orbit. The demo analyzes $\text{binomial}(n,2)$ on indices zero through 100, counts positive square-triangular sizes below 10^{100} , applies a root residue condition and selects the first twelve sizes by rank.

The Gamma analysis tests power values of the chosen expression on its domain. The size selector addresses complete Pell-orbit objects with size or residue constraints. It does not infer that unrelated Gamma expressions share that family. Bounds limit output growth and modular state work.

This supports compatible combinatorial capacities and exact downstream size selection. Earlier sourced factorial-ratio unit domains remain in receipts/populations. Unit conditions on uncanceled recurrence factors are not a global classification of factorial-ratio power values.

Held-out mathematical task protocols

The generator emits 256 recover-rank tasks from original points of $2x^2=3y^3$ with y at most 6 times 10^{12} . A deterministic hash partitions parameter groups into train, validation and test. Opposite signed branches with the same parameter stay together. The split has 178 training tasks, 40 validation tasks and 38 test tasks.

Public files omit answers, private records and ranks. The private packet contains original identities and exact answers. Predictions map task IDs to exact answer objects. Missing predictions count as incorrect; unknown IDs are rejected and ranks must be genuine integers. The recorded evaluation is an oracle replay, not agent performance. The private packet is shipped for reproducibility and must be withheld during blind evaluation.

This supplies an auditable within-family parameter split and an evaluation harness. It does not establish unseen-family generalization. A future benchmark can hold out whole source families and add richer distributions.

Measured configuration tuning

The tuner accepts an `ExactPopulation`, a caller-owned Python kernel and an independently computed reference. It checks every warmup and measured output, shuffles candidate order each round, records nanosecond trials and selects measured median winners. All candidates must fit the evaluation budget. Tuning is a direct Python API; the service does not accept executable callbacks.

The actual workload multiplies deterministic 32 by 32 integer matrices. Four configurations use row tile n^3 and column tile n^2 for n from one through four. Thus row_tile^2 equals column_tile^3 ; tile capacity is n^5 . Partial edge tiles are supported. Seven interleaved measured rounds follow one warmup, producing 28 measured candidate trials. A separately timed Python dot-product implementation supplies the independent reference and conventional comparison.

The recorded run selected rank three in the compatible tile family. The conventional implementation was faster than the best tiled candidate on this host. That result is retained in `measured_tuning.json`. The new capability is measuring and selecting admissible configurations; this experiment does not demonstrate a conventional multiplication speed advantage. Python loops, timing noise and separately timed baselines limit the comparison. Compiled kernels, hardware and larger workloads require new measurements.

Reproduction and verification

```
export PYTHONPATH=python
python -m unittest discover -s python/tests -p test_applications.py
python python/develop_applications.py --output /tmp/application-replay
python -m perfectpower service --database /tmp/query-catalogue.sqlite \
  < receipts/applications/service_requests.jsonl
make test
```

The 21 new tests cover persistence, cache reuse, aliases, atomic failures, multiplicity-preserving joins, request recovery, process restart, sequence laws, experiments, calibration optima, exact graph distributions, rational orders, geometry, complete size selection, tuning contracts, a real kernel and grouped evaluation. The full existing Python suite also runs. Optional Lean replay skips retain their dependency scope.

Generated SQLite bytes are not committed because they are platform-specific. Portable definitions and the transcript reconstruct the database. Arithmetic and dataset receipts are deterministic. Hardware timings and the summary's timing fields vary. The complete source ZIP and this monograph accompany the update.

Remaining mathematics and deployment

The release implements the software integrations with tested APIs and demonstrated workloads. Broader effective arithmetic backends, a globally compatible chart atlas, the golden frame coefficient and the physical phase remain open. They require new mathematics or research evidence. Industrial models, compiled-kernel acceleration, unseen-family evaluation and production deployments remain application-specific follow-on work. The direct-use roadmap records these boundaries explicitly.